

Conception d'une station d'arrosage autonome sous ESP32

Projet réalisé dans le cadre de la formation en Ingénierie des Systèmes
Embarqués en 4e année
ELSE 4 FISA

Présenté par :

Ndeye Khady DIOP

Salma ELFIACHE

Wissal BELLAHCEN

Sous la direction de :

M. Christian PETER

1. Introduction générale

1.1 Contexte du projet

Dans un contexte mondial marqué par le dérèglement climatique, la gestion raisonnée de l'irrigation représente un enjeu croissant. L'arrosage manuel présente des limites importantes: ils ne tiennent pas compte des conditions météorologiques réelles et conduisent fréquemment à une sur-irrigation néfaste pour l'environnement.

Ce projet s'inscrit dans le cadre du module d'électronique, visant à concevoir un système d'arrosage automatique intelligent, capable de piloter jusqu'à quatre zones d'irrigation de façon indépendante, tout en s'adaptant aux conditions extérieures mesurées en temps réel (température, humidité de l'air, humidité du sol, pluviométrie) .

1.2 Problématique

Comment concevoir un système d'arrosage capable de :

- Commander jusqu'à 4 électrovannes de façon indépendante,
- S'adapter en temps réel aux conditions météorologiques (pluie, température, humidité),
- Fonctionner sur batterie avec une autonomie maximale,
- Afficher clairement les informations à l'utilisateur

1.3 Objectifs et fonctionnalités attendues

- Pilotage de 4 vannes motorisées 12V (commande impulsionnelle).
- Planification horaire : heure de début et fréquence (en heures) réglables.
- Adaptation automatique : modulation de la durée d'arrosage selon la température et l'humidité de l'air ; annulation en cas de pluie ou de risque de gel.
- Mesure et affichage : température, humidité relative de l'air, humidité du sol (sonde analogique), cumul pluviométrique quotidien, état des vannes.

- Interface utilisateur : écran TFT couleur (ILI9341), avec affichage de logos et de textes.
- Alimentation sur batterie (3,7V Li-ion) avec convertisseur élévateur 12V pour les vannes.
- Économie d'énergie : utilisation du mode **deep sleep** et du coprocesseur ULP pour le comptage de la pluie.

1.4 Cahier des charges (exigences et contraintes)

1.4.1 Description générale

Le système gère 4 circuits d'irrigation indépendants via 4 électrovannes. Les vannes peuvent être activées ou désactivées ; elles sont déclenchées successivement pour éviter une chute de débit. La durée d'arrosage est individuelle par zone, la fréquence est commune. Les mesures des capteurs, la date et l'heure sont affichées en temps réel.

1.4.2 Contraintes générales

Domaine	Exigence
Alimentation	Batterie 3,7V + convertisseur 12V pour vannes.
Étanchéité	Boîtier IP65 (extérieur).
Coût	Total <150 € (composants + PCB).
Autonomie	Deep sleep majoritaire, consommation <10 µA en veille.
Reproductibilité	Schéma Eagle, code commenté, BOM fournie.

Simplicité	Pas de dépendance à un serveur distant, paramètres modifiables dans le code.
Interfaces	Compatibilité avec l'ESP32

1.4.3 Spécifications fonctionnelles

- Vannes : commande impulsionnelle (5s) non-bloquante ; pas d'ouverture simultanée.
- Planification : heures de début fixes (ex. 20h00, 20h30...), fréquence en heures (calcul par modulo jours), durée individuelle (minutes).
- Capteurs : DHT22 (température, humidité), pluviomètre à auget (0,2794 mm/basculement), sonde d'humidité du sol (analogique, 0–100%).
- RTC : DS3231 (I2C, pile CR2032 de secours).
- Affichage : écran TFT ILI9341 (SPI), rafraîchissement par flags, logos PNG.
- Adaptation : annulation si pluie >5 mm, $T^{\circ} < 4^{\circ}\text{C}$, ou sol $\geq 60\%$; modulation durée : +30% si $T^{\circ} \geq 30^{\circ}\text{C}$, -20% si humidité air $\geq 80\%$, +20% si sol $\leq 20\%$.
- Deep sleep : entrée après 120 secondes d'inactivité (pas d'impulsion pluie, pas d'arrosage). Réveil toutes les 5 minutes. ULP actif pour compter la pluie.

(Les spécifications techniques détaillées des composants sont reportées dans l'état de l'art, section 2.)

2. État de l'art et choix technologiques

2.1 Choix de l'unité de traitement – ESP32-WROOM-32D

L'ESP32 a été sélectionné pour ses caractéristiques :

- Double cœur Xtensa LX6 à 240 MHz, puissance suffisante pour l'affichage et les algorithmes.
- Coprocesseur ULP (Ultra Low Power) exécutant un programme en assembleur même en deep sleep – indispensable pour compter les impulsions du pluviomètre sans réveiller le CPU.
- Consommation : <10 μ A en deep sleep (ULP actif).
- Interfaces : SPI (écran), I2C (RTC), ADC (sol), GPIO (vannes).
- Wi-Fi/Bluetooth intégrés pour évolutions futures.

Caractéristiques principales : SRAM 520 Ko, Flash 4 Mo, 38 GPIO, ADC 12 bits, alimentation 3,0–3,6 V.

2.2 Sélection des capteurs et actionneurs

Type	Modèle	Précision / Sensibilité	Interface
Température / humidité	DHT22	$\pm 0,5^{\circ}\text{C}$, $\pm 2\%$ HR	1-wire (GPIO14, pull-up 4,7k Ω)
Pluviomètre	Lexca001	0,2794 mm/basculement	Contact sec, GPIO34 (pull-up externe 10k Ω)
Humidité du sol	Sonde capacitive	0–100% (calibration)	Analogique, GPIO35
Électrovanne	ACARPS IP65 (3 fils)	12V DC, 6W, commande impulsionnelle 5s	2 GPIO par vanne (ouverture/fermeture) via MOSFET

Détail des vannes : type *latching* (maintien de position sans courant après impulsion). Pilotage par N-MOS BSS123 (côté ESP32) et P-MOS DMP3098 (côté 12V). Résistances de 330 Ω sur les grilles.

2.3 Gestion de l'énergie

L'autonomie sur batterie repose sur trois axes complémentaires :

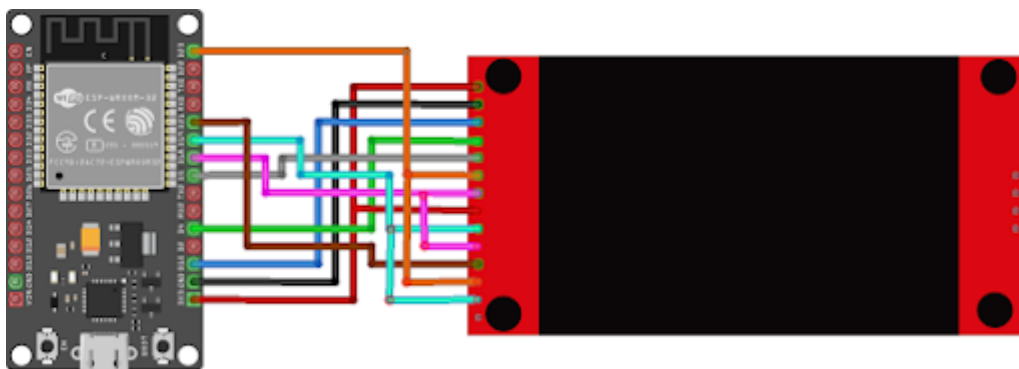
- Architecture d'alimentation : batterie Li-ion 18650 (3,7V, 3000 mAh). Alimentation directe de l'ESP32 (régulateur interne). Un convertisseur élévateur MT3608 fournit le 12V aux vannes. Une diode Schottky SS34 protège contre les inversions de polarité.
- Vannes latching : consommation uniquement pendant l'impulsion de 5 secondes (≈ 30 J par action), puis zéro.
- Mode deep sleep de l'ESP32 : le CPU est éteint la plupart du temps, seule l'ULP tourne. Consommation < 10 μ A. L'entrée en veille est conditionnée par l'absence d'arrosage et un délai d'inactivité de 120 secondes. Réveil périodique toutes les 5 minutes.

2.4 Interface homme-machine – Écran TFT ILI9341

Écran 2,4 pouces, 240×320 pixels, 16 bits, piloté en SPI.

Brochage utilisé : MOSI=23, MISO=19, SCK=18, CS=15, DC=5, RST=4, LED=13.

Logos PNG (température, humidité, pluie, terre) stockés en mémoire Flash (fichier images.h) et décodés par la bibliothèque PNGdec.



Shémat de cablage provenant du site :

<https://electroniqueamateur.blogspot.com/2021/05/utilisation-dun-ecran-tactile-tft-spi.html>

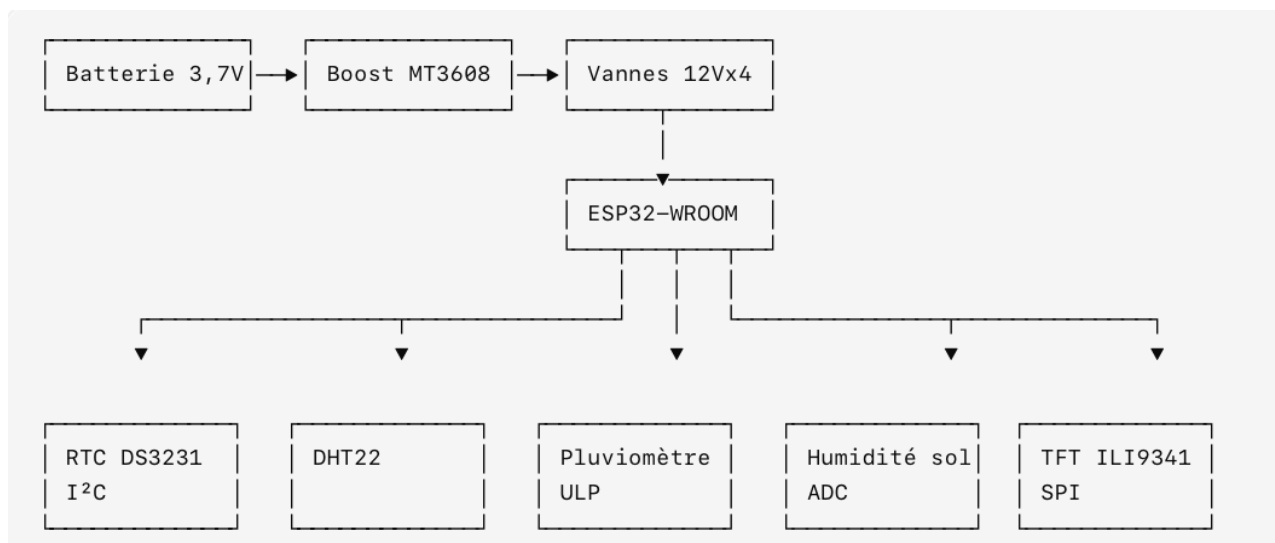
Voici le branchement de l'écran sur le module ESP32:

- Broche VCC de l'écran: sortie 3,3 V de l'ESP32
- Broche GND de l'écran: broche GND de l'ESP32
- Broche CS de l'écran: broche D15 de l'ESP32
- Broche Reset de l'écran: broche D4 de l'ESP32
- Broche DC de l'écran: broche D5 de l'ESP32
- Broche SDI (MOSI) de l'écran: broche D23 de l'ESP32
- Broche SCK de l'écran: broche D18 de l'ESP32
- Broche LED de l'écran: sortie 3,3 V de l'ESP32
- Broche CDCK (MISO) de l'écran: broche D19 de l'ESP32
- Broche T_CLK de l'écran: broche D18 de l'ESP32
- Broche T_CS de l'écran: broche D21 de l'ESP32
- Broche T_DIN de l'écran: broche D23 de l'ESP32
- Broche T_DO de l'écran: broche D19 de l'ESP32
- Broche T_IRQ de l'écran: pas branchée

3. Architecture globale du système

3.1 Synoptique matériel

Le système repose sur un ESP32-WROOM assurant l'acquisition des données environnementales, leur traitement et la commande des électrovannes. L'alimentation des actionneurs est assurée par une conversion DC-DC élévatrice, tandis que l'interface utilisateur est réalisée via un écran TFT SPI



3.2 Modes de fonctionnement

Le système opère selon deux modes distincts, gérés dynamiquement en fonction de l'activité détectée.

Mode actif. À chaque démarrage ou réveil, l'ESP32 initialise l'ensemble des périphériques (RTC DS3231, écran TFT, capteur DHT22) et lance la boucle principale. Celle-ci exécute quatre tâches non-bloquantes par des timers millis() : la mise à jour de l'affichage à la seconde, la lecture des capteurs toutes les 5 secondes, la lecture du compteur pluviométrique toutes les 2 secondes, et la vérification du planning d'arrosage à chaque seconde. L'affichage est actif et l'interface utilisateur réactive aux appuis sur les boutons.

Mode veille profonde (Deep Sleep). Lorsque les conditions du garde-fou sont réunies, l'ESP32 sauvegarde les données persistantes (cumul de pluie, timestamps de fin d'arrosage, horodatage de la dernière pluie) dans la mémoire RTC — la seule zone mémoire qui survit au Deep Sleep — puis entre en sommeil. Un réveil périodique par timer permet de vérifier régulièrement si un arrosage planifié doit être déclenché. Lors du réveil, le système distingue un premier démarrage (remise à zéro de tous les compteurs et initialisation de l'ULP) d'un réveil depuis la veille (restauration des valeurs mémorisées), avant de reprendre la boucle principale.

4. Conception logicielle

4.1 Architecture Orientée Objet (POO)

Le code est découpé en plusieurs classes pour faciliter la maintenance et l'évolution :

- *Case (classe abstraite)* : représente une zone rectangulaire de l'écran, avec position, taille, couleur de fond, et un flag de modification.
- *SensorCase* : hérite de *Case*, ajoute une valeur entière, une unité, un logo PNG, et la méthode `draw()` pour afficher ces informations.
- *VanneCase* : hérite de *Case*, ajoute un nom, une fréquence (heures), une durée (minutes), l'état (ouverte/fermée) et les GPIO de commande.
- *Interface* : gère l'écran global, l'affichage de l'heure, et appelle le dessin des cases.
- *SensorManagement* : regroupe la lecture des capteurs (DHT22, RTC, ULP, sol) et met à jour les objets *SensorCase* correspondants.

Cette modularité permet d'ajouter facilement un nouveau capteur ou une nouvelle vanne.

4.2 Optimisation de l'Interface Homme-Machine

Affichage par flags : chaque case possède un booléen flag qui passe à 1 quand sa valeur ou sa couleur change. La fonction *Update()* ne redessine que les cases dont le flag est à 1, puis le remet à 0. Cela évite de rafraîchir tout l'écran à chaque seconde et réduit la charge CPU.

Logos PNG : les icônes (température, humidité, pluie, terre) sont stockées en mémoire Flash (via `images.h`) et décodées avec la bibliothèque PNGdec. Elles sont dessinées une fois lors de l'affichage initial.

Horloge : l'heure est lue depuis le RTC et affichée dans le bandeau supérieur mise à jour chaque seconde.

Couleurs : les arrière-plans des capteurs changent selon les seuils (rouge = critique, orange/jaune = attention, blanc = normal). Les vannes sont vertes (ouvertes), rouges (désactivées), blanches (fermées).

4.3 Logique d'arrosage asynchrone

Algorithme d'arrosage automatique — Description détaillée

1. Démarrage du système (phase d'initialisation)

Lorsque l'ESP32 est alimenté ou sort d'une période de veille, le système détermine d'abord la cause du démarrage :

- Premier démarrage (alimentation mise en place):

- Le programme ULP est initialisé dans le coprocesseur basse consommation, chargé de surveiller en permanence le pluviomètre.
- L'ensemble des compteurs est remis à zéro : pluie totale (0 mm), horodatages de fin d'arrosage (0), dernier horodatage de pluie (0).

- Réveil après une période de veille profonde (réveil programmé par temporisateur) :

- Les valeurs conservées avant la mise en veille (pluie totale, horodatage de fin d'arrosage, horodatage de la dernière pluie) sont récupérées depuis la mémoire RTC, qui préserve les données pendant le *deep sleep*.

Dans tous les cas de figure, l'initialisation suivante est effectuée :

- Horloge RTC (module DS3231 via le bus I2C)
- Écran TFT
- Capteur DHT22 (mesure de la température et de l'humidité de l'air)
- Capteur d'humidité du sol (lecture analogique par CAN)
- Affichage principal des valeurs courantes

2. Boucle principale

La boucle principale s'exécute en continu et gère cinq tâches concurrentes, non bloquantes, chacune pilotée par des temporisateurs basés sur millis() :

- Tâche A — Mise à jour de l'affichage (périodicité : 1 seconde)

Le gestionnaire d'écran vérifie les zones dont le drapeau de mise à jour est positionné à 1 (valeur modifiée) et ne redessine que ces zones. L'heure est également rafraîchie.

- Tâche B — Lecture des capteurs (périodicité : 5 secondes)

Le capteur DHT22 est interrogé pour obtenir la température (°C) et l'humidité relative de l'air (%). Ces valeurs sont converties en entiers puis stockées dans les zones correspondantes. En cas de changement de valeur, le drapeau de mise à jour est activé, ce qui déclenchera un rafraîchissement de l'affichage au cycle suivant.

- Tâche C — Lecture du pluviomètre (périodicité : 2 secondes)

Pendant l'exécution du programme principal, le coprocesseur ULP comptabilise les basculements du pluviomètre. Toutes les 2 secondes, le compteur est lu et remis à zéro. Chaque basculement correspond à une hauteur de pluie de 0,2794 mm.

En cas de nouvelles impulsions détectées :

- La quantité de pluie correspondante est ajoutée au total journalier.
- L'horodatage exact (en secondes Unix, fourni par le RTC) de cette détection est enregistré, afin de déterminer ultérieurement s'« il pleut actuellement » lors de la prise de décision d'arrosage.
- L'affichage de la zone réservée au pluviomètre est mis à jour.

- Tâche D — Lecture de l'humidité du sol (périodicité : 10 secondes)

Le capteur d'humidité du sol est lu via le convertisseur analogique-numérique de l'ESP32. La valeur analogique brute est convertie en pourcentage (0 % = sol sec, 100 % = sol saturé). En cas de modification de la valeur, le drapeau de mise à jour est activé, et l'affichage correspondant sera actualisé au cycle suivant.

- Tâche E — Vérification du planning d'arrosage (périodicité : 1 seconde)

Cette tâche constitue l'élément central de l'algorithme. Elle est détaillée ci-après.

3. Logique de décision d'arrosage (gererVannesAutomatiques)

Cette fonction est appelée chaque seconde, avec l'heure exacte fournie par le RTC. Elle traite indépendamment les quatre vannes, dans l'ordre de la vanne V1 à la vanne V4.

Pour chaque vanne, deux types de vérifications sont effectués :

- **Vérification de la condition d'ouverture (déclenchement)**

Condition de déclenchement horaire :

L'heure courante doit correspondre exactement à l'heure et à la minute programmées pour cette vanne, et la seconde doit être zéro.

Exemple : V1 se déclenche à 20:00:00, V2 à 20:30:00, etc.

Chaque vanne possède une fréquence exprimée en heures (par exemple, 48 h = un arrosage tous les deux jours). Le nombre de jours écoulés depuis le 1^{er} janvier 1970 (timestamp Unix divisé par 86 400) est calculé. Si ce nombre de jours est divisible par l'intervalle programmé, le jour est considéré comme un jour d'arrosage pour cette vanne. Dans le cas contraire, on passe à la vanne suivante.

Si le jour est un jour d'arrosage, les gardes météorologiques sont vérifiées :

- **Garde 1 — Pluie accumulée excessive**

Si le total des précipitations enregistrées depuis le début de la journée atteint ou dépasse 5 mm, l'arrosage est annulé (le sol a déjà reçu un apport naturel suffisant).

- **Garde 2 — Pluie en cours**

On détermine si une pluie a été détectée au cours des 15 dernières minutes. Pour cela, l'horodatage RTC actuel est comparé au dernier horodatage enregistré lors d'une détection d'impulsion du pluviomètre. Si la différence est inférieure à 900 secondes (15 min), on considère qu'il pleut encore, et l'arrosage est annulé.

- **Garde 3 — Risque de gel**

Si la température actuelle est inférieure ou égale à 4 °C, l'arrosage est annulé afin d'éviter le gel de l'eau dans les canalisations et les détériorations associées.

- **Garde 4 — Sol déjà suffisamment humide**

Si l'humidité mesurée du sol est supérieure ou égale à 60 %, l'arrosage est annulé : le sol contient déjà une quantité d'eau adéquate, et un arrosage supplémentaire serait superflu, voire risquerait d'asphyxier les racines. Si l'ensemble des gardes est franchi, le système procède au **calcul de la durée d'arrosage** :

La durée de base est celle configurée individuellement pour chaque vanne (en minutes). Cette durée est ensuite modulée par les facteurs suivants :

- Modulateur 1 — Température élevée

- Température $\geq 35\text{ °C}$: durée $\times 1,50$ (+50 %) — condition caniculaire
 - Température comprise entre 30 °C et 35 °C (exclu) : durée $\times 1,30$ (+30 %)
- forte chaleur

- Modulateur 2 — Humidité de l'air élevée

- Humidité relative $\geq 80\%$: durée $\times 0,80$ (-20 %) — air saturé, évaporation réduite

La durée finale est arrondie à l'entier inférieur, avec un minimum d'une minute.

- Modulateur 3 — Sol très sec

- Humidité du sol $\leq 20\%$: durée $\times 1,20$ (+20 %) — sol sec, besoin hydrique accru.

La durée finale est arrondie à l'entier inférieur, avec un minimum d'une minute.

● **Déclenchement effectif :**

- La vanne est ouverte (envoi d'une impulsion sur la broche GPIO pendant 5 secondes, de manière non bloquante).
- La date et l'heure exactes de fermeture prévue sont calculées :
 $\text{timestamp_fin} = \text{timestamp_actuel} + (\text{durée_finale} \times 60 \text{ secondes}).$

Cette valeur est conservée en mémoire RTC afin de résister à une éventuelle mise en veille.

● **Vérification de la fermeture**

À chaque seconde, pour chaque vanne dont l'état est « ouverte » (state = vrai), l'horodatage RTC courant est comparé à l'horodatage de fermeture programmé.

Si l'heure de fin est atteinte ou dépassée, la vanne est fermée (envoi d'une impulsion inverse sur la broche GPIO pendant 5 secondes).

4. Remise à zéro quotidienne à minuit

Chaque nuit, à 00:00:00, le système procède une seule fois aux remises à zéro suivantes :

- Total journalier des précipitations
- Horodatage de la dernière pluie détectée
- Affichage associé au pluviomètre

5. Gestion des vannes (mode non bloquant)

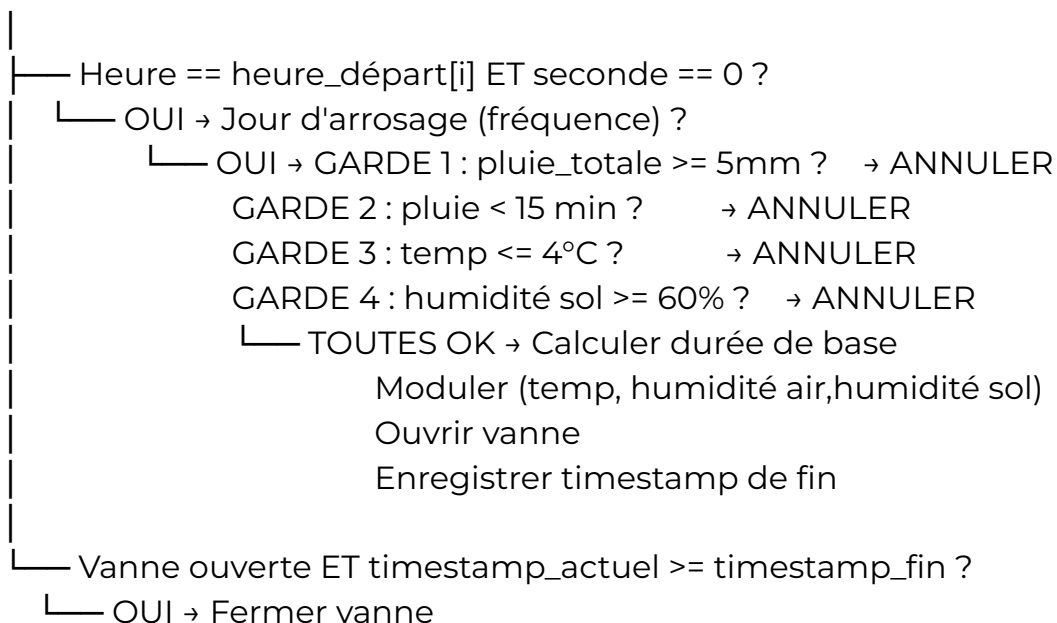
Les vannes utilisées (modèle ACARPS IP65) sont de type *latching* (motorisées, trois fils) : elles conservent leur position sans consommation de courant une fois actionnées.

Pour les commander, une impulsion de 5 secondes est envoyée sur la broche GPIO correspondante.

Le surveillant `checkTimer()` est appelé à chaque cycle de la boucle principale. Si une impulsion est en cours depuis plus de 5 secondes, il désactive les deux broches GPIO et libère la vanne, sans jamais bloquer le déroulement du programme.

Résumé visuel

CHAQUE SECONDE



4.4 Le coprocesseur ULP

Le coprocesseur ULP (*Ultra Low Power*) est un processeur auxiliaire intégré à l'ESP32, capable d'exécuter un programme en assembleur pendant que les cœurs principaux sont en Deep Sleep. Sa consommation est de l'ordre de quelques centaines de microampères, ce qui le rend indispensable pour surveiller un signal externe de façon continue sans vider la batterie.

Rôle dans le système:

L'ULP est exclusivement chargé du comptage des impulsions du pluviomètre, connecté sur GPIO34. Le capteur à auget Lexca001 génère un front à chaque basculement, chaque impulsion correspondant à 0,2794 mm de pluie. Surveiller ce signal depuis le cœur principal serait incompatible avec le mode Deep Sleep ; l'ULP prend donc en charge cette tâche de façon transparente.

Configuration de la broche:

Dans la fonction **initULP()**, GPIO34 est initialisée en entrée RTC via **rtc_gpio_init()** et **rtc_gpio_set_direction()**. Les résistances de pull-up et pull-down internes sont explicitement désactivées car une résistance externe de 10 kΩ assure le niveau logique au repos. La fonction **rtc_gpio_hold_en()** verrouille l'état de la broche pendant le Deep Sleep pour éviter toute dérive.

Programme assembleur ULP:

Le programme est écrit sous forme de tableau d'instructions `ulp_insn_t` et chargé en mémoire via `ulp_process_macros_and_load()`. Il s'exécute en boucle toutes les 50 ms (période configurée par `ulp_set_wakeup_period(0, 50000)`). La logique de détection de front repose sur la comparaison entre l'état courant de la broche et le dernier état mémorisé. Seuls les fronts montants (transition 0→1) incrémentent le compteur, ce qui évite de compter deux fois chaque basculement. Les valeurs sont stockées dans RTC_SLOW_MEM aux adresses 20 (ULP_COUNT_ADDR) et 21 (ULP_LAST_STATE_ADDR), choisies suffisamment hautes pour ne pas entrer en conflit avec le reste du programme et éviter tout crash.

Persistance entre les cycles de veille.

Lors d'un premier démarrage (cause de réveil $\neq$ ESP_SLEEP_WAKEUP_TIMER), les deux cellules mémoire sont remises à zéro et l'état initial de la broche est supposé à 1 (niveau haut au repos). Lors d'un réveil depuis le Deep Sleep, ces valeurs sont conservées intactes, garantissant qu'aucune impulsion n'est perdue pendant la veille.

Lecture depuis le programme principal. La fonction getPulseCount() lit le compteur via RTC_SLOW_MEM[ULP_COUNT_ADDR] & 0xFFFF, remet immédiatement la case à zéro, et retourne le nombre d'impulsions accumulées depuis la dernière lecture. Ce masque 16 bits évite de lire des bits de statut parasites stockés dans les bits hauts de la mémoire RTC. Cette fonction est appelée toutes les 2 secondes par la tâche C de la boucle principale pour mettre à jour le cumul pluviométrique journalier.

4.5 Stratégie énergétique (Deep Sleep)

L'ESP32 est programmé pour entrer en mode Deep Sleep afin de préserver la batterie. Cependant, un algorithme de sécurité a été implémenté pour empêcher une mise en veille critique : le système ne s'endort que si **deux conditions strictes** sont réunies :

1. Aucune vanne n'est actuellement en cours d'ouverture, pour éviter qu'une vanne ne reste bloquée ouverte pendant le sommeil.
2. Un délai d'inactivité de 120 secondes s'est écoulé sans modification de l'interface (pas de pluie détectée, pas de manipulation).

5. Conception matérielle

5.1 Saisie du schéma électronique

Le schéma a été réalisé sous Eagle 9.6.2 et il comprend les blocs suivants:

- Alimentation : batterie 3,7V $\rightarrow$ boost MT3608 $\rightarrow$ 12V pour vannes. Diode Schottky (D1) pour protection, self 22 μ H, condensateurs de filtrage.
- ESP32 : module WROOM-32, avec connecteur pour programmation série (DOWNLOAD : TX, RX, IO0, RST, 3V3, GND).

- Capteurs : DHT22 (pull-up 4,7k Ω sur GPIO14), pluviomètre sur GPIO34 (pull-up externe 10k Ω à ajouter), humidité sol sur GPIO35 (à câbler).
- RTC : DS3231, alimenté par la pile CR2032 (VBAT) et par le 3,3V (VCC). I2C avec résistances 4,7k Ω .
- Vannes : 4 canaux, chaque vanne pilotée par deux transistors MOSFET : un BSS123 (N-MOS) côté ESP32, et un P-MOS (type DMP3098) côté 12V. Des résistances de limitation (330 Ω) sont placées sur les grilles des N-MOS.
- Écran TFT : connecté en SPI (MOSI, MISO, SCK, CS, DC, RST, LED).
- Connecteurs : BATTERIE (1x5), DOWNLOAD (1x6), JP12 (1x5 pour I2C et 3,3V), VANNE_1..4 (1x3).

5.2 Placement et routage (Layout)

Le fichier .brd contient un début de placement et de routage. Les pistes ne sont pas encore finalisées (de nombreuses connexions sont à l'état Unrouted). Les axes d'amélioration sont :

- Terminer le routage avec des largeurs adaptées.
- Ajouter des plans de masse (polygones) pour réduire les interférences.
- Placer les résistances de pull-up (pluviomètre, I2C) près des broches correspondantes.
- Vérifier l'encombrement pour un boîtier IP65 (dimensions 150×100×50 mm).

5.3 État d'avancement matériel

- Prototype sur breadboard : validé pour l'ESP32, le TFT, le DHT22, le RTC . Le pluviomètre a été testé avec une simulation manuelle.
- PCB : non encore fabriqué et les vannes pas encore testées

6. Tests et validations

La phase de test a été réalisée sur un prototype fonctionnel assemblé sur *breadboard*. L'objectif de cette étape était de valider l'acquisition des

données, la stabilité de l'architecture logicielle, la gestion énergétique, et de simuler la commande des actionneurs avant l'intégration finale sur le circuit imprimé.

6.1 Validation de l'acquisition des données et de l'IHM

- **Capteurs environnementaux** : La lecture du capteur DHT22 (température et humidité de l'air) et de la sonde analogique (humidité du sol via le convertisseur ADC de l'ESP32) a été validée. Les valeurs remontent correctement et les couleurs de l'interface graphique (IHM) s'adaptent dynamiquement aux seuils critiques (ex: passage au rouge si le sol est très sec).
- **Horloge RTC** : La communication I2C fonctionne parfaitement. Le module conserve l'heure exacte même lors des redémarrages ou des phases de *Deep Sleep* grâce à sa pile de sauvegarde.
- **Interface graphique** : L'optimisation logicielle par *flags* a prouvé son efficacité. L'écran ne scintille pas, car seules les zones dont les données ont changé sont redessinées, économisant ainsi les ressources du microcontrôleur.

6.2 Simulation et validation de la logique des vannes

Bien que les tests de puissance sous 12V et les essais hydrauliques réels des électrovannes n'aient pas encore pu être réalisés, **la logique de commande a été intégralement validée au niveau logiciel** :

- **Planification et Modulateurs** : Le moniteur série a permis de vérifier que l'algorithme calcule correctement les jours d'arrosage. Les modulations de durée et les annulations s'exécutent conformément au cahier des charges.

6.3 Validation énergétique (*Deep Sleep* et ULP)

Le système de gestion de l'énergie a été l'un des points les plus critiques à tester :

- **Chronomètre d'inactivité** : L'ESP32 bascule avec succès en sommeil profond (*Deep Sleep*) après 120 secondes d'inactivité, à condition qu'aucune vanne ne soit en cours d'ouverture.
- **Comptage ULP** : Le coprocesseur ULP reste fonctionnel pendant le sommeil du processeur principal. Les basculements du pluviomètre simulés pendant la veille ont été correctement récupérés et additionnés au réveil du système.

7. Conclusion et perspectives

Ce projet a permis de concevoir et de développer un système d'arrosage intelligent basé sur l'ESP32, répondant aux objectifs fixés en début d'étude. L'architecture matérielle et logicielle a été entièrement définie, depuis la conception du schéma électronique jusqu'à la réalisation d'un prototype fonctionnel sur breadboard.

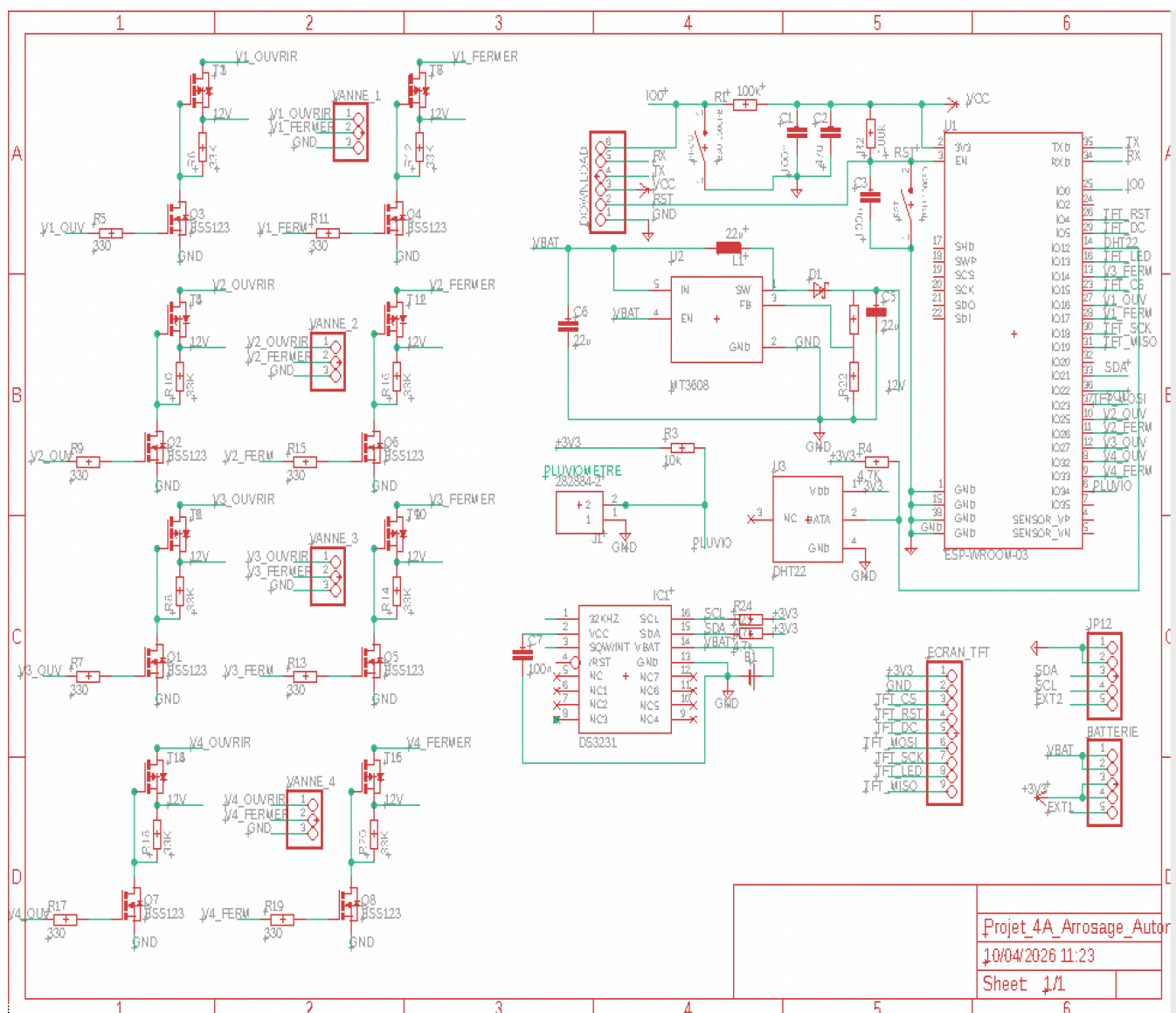
Les principales fonctionnalités attendues ont été validées, notamment la planification automatique de l'arrosage, l'adaptation du fonctionnement en fonction des conditions météorologiques (température, humidité de l'air et du sol), ainsi que la commande impulsionnelle et non-bloquante de plusieurs électrovannes. L'intégration du coprocesseur ULP en langage assembleur pour le comptage pluviométrique, couplée à la mise en œuvre effective du mode *deep sleep*, constitue un apport technique majeur. Cela permet d'optimiser drastiquement la consommation énergétique tout en assurant une surveillance continue. Par ailleurs, l'architecture logicielle orientée objet mise en place facilite grandement la maintenance, la lisibilité du code et l'ajout futur de nouvelles fonctionnalités. Les différentes contraintes rencontrées au cours du développement, telles que la gestion de la mémoire partagée de l'ULP, la création d'un chronomètre d'inactivité ou les conflits matériels des broches, ont également été identifiées puis rigoureusement corrigées.

Certaines étapes restent toutefois à finaliser pour aboutir à un produit totalement abouti. La finalisation du routage et la fabrication d'un circuit imprimé constitueront une étape essentielle vers une version plus robuste et industrialisable du prototype, permettant de tester le pilotage de puissance réel des vannes sous 12V. En l'état, il constitue d'ores et déjà une

solution complète, autonome et fonctionnelle, prête à affronter les conditions réelles d'utilisation.

Annexes

Annexe A – Schéma électrique



Annexe B – Code source complet

Dépôt Git contenant les fichiers .ino, .h, .cpp

Annexe C – Liste des bibliothèques utilisées

Bibliothèque	Version	Utilisation
TFT_eSPI	2.5.0	Pilotage écran ILI9341
PNGdec	1.1.1	Décodage PNG (logos)
RTCLib	2.1.0	Gestion RTC DS3231
DHT sensor library	1.4.4	Lecture DHT22
Adafruit_Sensor	1.1.6	Interface capteurs (dépendance)
Wire, SPI, esp32/ulp.h, driver/rtc_io.h	(core ESP32)	I2C, SPI, ULP, deep sleep

Annexe D – Liste du matériel

Référence	Désignation	Qté
ESP-WROOM-32	Module ESP32	1
MT3608	Boost 12V	1
DS3231	Module RTC avec pile	1
DHT22	Capteur T/H	1
Pluviomètre	Lexca001 (auget)	1
Vanne ACARPS	12V 3 fils	4
BSS123	N-MOS SOT23	8
DMP3098 (ou équivalent)	P-MOS SOT23	8
Résistances 330Ω, 10kΩ, 4,7kΩ	divers	20
Condensateurs 22μF, 100nF	0805	6
Self 22μH	0805	1
Diode Schottky SS34	DO-214	1
Écran TFT ILI9341	SPI 2,4"	1
Connecteurs 1x3, 1x5, 1x6	pitch 2,54 mm	6
Batterie 18650	3,7V	1
Boîtier IP65	150×100×50 mm	1